

NonPro's Team — Môn học Kiểm thử phần mềm

TÀI LIỆU MÔ TẢ HỆ THỐNG

System Description Document — Green Juice Hub

Dự án: Green Juice Hub — Website thương mại điện tử nước ép & smoothie

Phiên bản: 1.0

Ngày biên soạn: 20/08/2026

Người biên soạn: Hiền Trần (Trưởng nhóm)

Nhóm: NonPro's Team — Hiền, Nhân, Huy, Vũ, Thiện

Phương pháp: Biên soạn từ khảo sát trực tiếp mã nguồn (reverse-engineered)

Tài liệu liên quan: Dùng song hành với Tài liệu SRS (đặc tả yêu cầu)

Mục lục

(Mở bằng Microsoft Word: bấm chuột phải vào bảng dưới đây → Update Field, hoặc Ctrl+A rồi F9, để mục lục tự cập nhật số trang)

Mục lục	2
1. Giới thiệu	4
2. Kiến trúc tổng thể hệ thống	5
3. Công nghệ sử dụng	5
3.1 Frontend	5
3.2 Backend	6
3.3 Cơ sở dữ liệu & hạ tầng	6
4. Mô hình dữ liệu	7
4.1 Nhóm Xác thực (3 bảng)	7
4.2 Nhóm Địa chỉ (1 bảng)	8
4.3 Nhóm Sản phẩm (8 bảng)	8
4.4 Nhóm Giỏ hàng (2 bảng)	11
4.5 Nhóm Đơn hàng & Thanh toán (3 bảng)	12
4.6 Nhóm Khuyến mãi (2 bảng)	13
4.7 Nhóm Đánh giá (1 bảng)	14
4.8 Nhóm Khác (3 bảng)	15
5. Kiến trúc Backend	17
5.1 Cấu trúc package	17
5.2 Luồng xác thực & phân quyền (JWT)	17
5.3 Xử lý ngoại lệ tập trung	18
5.4 Tác vụ nền theo lịch (Scheduled Jobs)	18
5.5 Cấu hình & khóa bí mật	19
6. Kiến trúc Frontend	20
6.1 Cấu trúc thư mục	20
6.2 Định tuyến (React Router 7)	20
6.3 Quản lý trạng thái (Zustand)	21
6.4 Lớp gọi API & phiên đăng nhập (axiosConfig.js)	22
6.5 Trang & thành phần chính	22
7. Tích hợp bên thứ ba	24
8. Luồng nghiệp vụ chính	25
8.1 Đăng ký / Đăng nhập bằng OTP	25
8.2 Đăng nhập bằng mật khẩu	25
8.3 Đăng nhập bằng Google OAuth2	25
8.4 Đặt hàng & áp dụng khuyến mãi (Checkout)	25
8.5 Thanh toán VNPay	26
8.6 Thanh toán "chuyển khoản / MoMo" (giả lập qua VietQR + SePay)	26
8.7 Vòng đời đơn hàng sau khi đặt	27

8.8 Đánh giá sản phẩm sau khi nhận hàng.....	27
9. Bảo mật hệ thống.....	28
10. Triển khai.....	29
11. Hạn chế đã biết & khuyến nghị cải thiện	30
11.1 AdminTagController thiếu ràng buộc theo vai trò.....	30
11.2 Hai DTO/endpoint đổi mật khẩu trùng lặp	30
11.3 "MoMo" không phải tích hợp API MoMo thật.....	30
11.4 GHN chỉ hỗ trợ giao hàng nội thành Hồ Chí Minh	30
11.5 product_tags.tag không ràng buộc khoá ngoại	30
11.6 Chưa rõ khuôn dạng phản hồi lỗi 403 do phân quyền	31
11.7 Token JWT lưu ở localStorage thay vì cookie HttpOnly.....	31
11.8 SettingsPage: 2 chức năng chưa nối API backend	31
11.9 Yêu cầu phiên bản Java chưa nhất quán với README.....	31
11.10 Thiếu tài liệu API tự sinh (Swagger/OpenAPI).....	31
11.11 AutoDeliverScheduler suy đoán giao hàng thành công theo thời gian	31
11.12 JwtAuthFilter bỏ qua hoàn toàn xác thực trên /api/auth/logout	32
12. Phụ lục.....	33
12.1 Số lượng file mã nguồn Backend theo package	33
12.2 Số lượng file mã nguồn Frontend theo thư mục.....	33
12.3 Bảng thuật ngữ viết tắt.....	33

1. Giới thiệu

Tài liệu này mô tả kiến trúc kỹ thuật và cách vận hành thực tế của hệ thống Green Juice Hub, dùng song hành với Tài liệu SRS (đặc tả yêu cầu). Nếu SRS trả lời câu hỏi "hệ thống làm được gì", tài liệu này trả lời câu hỏi "hệ thống được xây dựng như thế nào" — phục vụ nhóm kiểm thử hiểu đúng cấu trúc mã nguồn để thiết kế test case ở đúng mức (unit/integration/system), xác định vùng ảnh hưởng khi có lỗi, và chuẩn bị môi trường kiểm thử.

Toàn bộ nội dung được biên soạn từ khảo sát trực tiếp mã nguồn dự án tại thời điểm 20/08/2026: schema.sql, 23 entity JPA, 26 controller, các lớp service, cấu hình bảo mật, và mã nguồn frontend (định tuyến, state, lớp gọi API).

2. Kiến trúc tổng thể hệ thống

Hệ thống theo kiến trúc client-server 3 lớp cổ điển, tách biệt hoàn toàn frontend và backend (giao tiếp qua REST API/JSON), triển khai trên 2 nền tảng cloud khác nhau:

Lớp	Thành phần	Vai trò
Trình diễn (Presentation)	Frontend React 19 (SPA) — triển khai trên Vercel	Giao diện người dùng, gọi REST API qua Axios, quản lý trạng thái phía client (Zustand)
Ứng dụng (Application)	Backend Spring Boot 4 — triển khai trên Railway	REST API, xử lý nghiệp vụ, xác thực/phân quyền, tích hợp bên thứ ba, tác vụ nền theo lịch
Dữ liệu (Data)	MySQL 8 (dữ liệu chính) + Redis (cache/rate-limit/blacklist token)	Lưu trữ bền vững 23 bảng dữ liệu quan hệ; Redis lưu dữ liệu tạm thời, có TTL

Luồng giao tiếp tổng quát: Trình duyệt người dùng → Frontend (Vercel, static hosting) → gọi REST API (JSON) qua HTTPS tới Backend (Railway) → Backend đọc/ghi MySQL và Redis, đồng thời gọi ra các dịch vụ bên thứ ba (Google, VNPAY, GHN, Cloudinary, Gemini) khi cần → Backend trả kết quả JSON → Frontend cập nhật giao diện. Ngoài chiều gọi này, hệ thống còn nhận request đến trực tiếp từ bên ngoài theo 2 kênh: (1) VNPAY gọi IPN/Return sau khi thanh toán, (2) SePAY gọi webhook khi phát hiện giao dịch chuyển khoản — cả hai đều đi thẳng vào Backend, không qua Frontend.

Frontend (Vercel) và Backend (Railway) khác domain, nên các luồng cần "quay lại trình duyệt" sau khi rời khỏi hệ thống (ví dụ thanh toán VNPAY) đều được thiết kế để VNPAY redirect thẳng về domain Frontend, sau đó Frontend mới gọi lại Backend để xác minh kết quả, thay vì Backend tự redirect (xem Mục 8.4).

3. Công nghệ sử dụng

3.1 Frontend

Thư viện / Công cụ	Phiên bản	Mục đích
React	^19.2.6	Thư viện dựng giao diện (SPA)
Vite	^8.0.12	Build tool / dev server
Tailwind CSS	^4.3.0	Styling utility-first, kèm @tailwindcss/typography
React Router DOM	^7.15.1	Định tuyến phía client
Zustand	^5.0.13	Quản lý trạng thái toàn cục (store)
Axios	^1.16.1	Gọi REST API, có interceptor tự refresh token
@react-oauth/google	^0.13.5	Đăng nhập Google OAuth2 phía client
Tiptap (+ extensions)	^3.26.1	Trình soạn thảo rich text (mô tả sản phẩm, chính sách)
Recharts	^3.8.1	Biểu đồ doanh thu ở trang Dashboard
Framer Motion	^12.40.0	Hiệu ứng chuyển động giao diện
lucide-react	^1.17.0	Bộ icon
react-markdown	^10.1.0	Hiển thị nội dung Markdown (câu trả lời AI tư vấn)

3.2 Backend

Thư viện / Công cụ	Phiên bản	Mục đích
Spring Boot	4.0.6	Nền tảng ứng dụng chính (parent POM)
Java	21	Ngôn ngữ lập trình (README ghi ≥ 17 , thực tế build yêu cầu 21 — xem Mục 11)
Spring Security	(theo Spring Boot 4)	Xác thực/phân quyền, cấu hình JwtAuthFilter tùy biến
Spring Data JPA	(theo Spring Boot 4)	ORM, repository, JPA Specification cho lọc động
jjwt (api/impl/jackson)	0.12.3	Sinh và xác thực JSON Web Token
Spring Data Redis	(theo Spring Boot 4)	Kết nối Redis cho OTP lock, chống brute-force, blacklist token
MySQL Connector/J	(theo Spring Boot 4)	Driver kết nối MySQL
Spring Boot Starter Mail + Thymeleaf	(theo Spring Boot 4)	Gửi email HTML (phản hồi liên hệ, thông báo) qua SMTP Gmail
Google API Client	2.4.0	Xác thực idToken của Google OAuth2
Lombok	—	Giảm mã lặp (getter/setter/builder) trên entity/DTO
Spring Boot Validation	(theo Spring Boot 4)	Validate DTO đầu vào bằng annotation (@NotBlank, @Size, @Pattern...)

Ghi nhận: pom.xml KHÔNG có springdoc-openapi/Swagger — hệ thống chưa có trang tài liệu API tự sinh; nhóm cần dùng Postman collection tự xây dựng để thay thế khi kiểm thử API (xem Mục 11).

3.3 Cơ sở dữ liệu & hạ tầng

- MySQL 8 — 23 bảng, ràng buộc khoá ngoại đầy đủ, một số bảng dùng kiểu ENUM ở tầng CSDL cho các cột trạng thái (đồng bộ với enum phía Java).
- Redis — lưu trạng thái tạm thời có thời hạn: giới hạn gửi OTP/số lần nhập sai OTP, giới hạn số lần đăng nhập sai mật khẩu, danh sách access token bị thu hồi (blacklist).
- Triển khai: Frontend trên Vercel (static hosting + CDN); Backend trên Railway (container, có hiện tượng cold start khi rảnh); MySQL/Redis cũng chạy trên hạ tầng phù hợp với Railway hoặc dịch vụ ngoài tương đương.

4. Mô hình dữ liệu

Cơ sở dữ liệu MySQL gồm 23 bảng (schema.sql, "Version: 2.0 | Synced with Java entities"), tương ứng 1-1 với 23 entity JPA phía backend. Toàn bộ bảng dùng engine InnoDB (hỗ trợ khoá ngoại và transaction). Mục này liệt kê đầy đủ cấu trúc cột của từng bảng, nhóm theo 8 miền nghiệp vụ giống Phụ lục B của Tài liệu SRS, để nhóm kiểm thử tra cứu ràng buộc dữ liệu khi thiết kế test case biên (boundary) và test case dữ liệu không hợp lệ.

Đa số khoá ngoại không khai báo rõ hành vi ON DELETE (mặc định RESTRICT của MySQL/InnoDB — không cho xoá bản ghi cha nếu còn bản ghi con tham chiếu). Các khoá ngoại có khai báo rõ ràng CASCADE hoặc SET NULL được ghi chú riêng trong từng bảng bên dưới — đây là điểm quan trọng cần kiểm thử khi thực hiện xoá dữ liệu (ví dụ: xoá user, xoá category).

4.1 Nhóm Xác thực (3 bảng)

users

Tài khoản người dùng — hỗ trợ 3 kiểu đăng nhập (mật khẩu, OTP điện thoại, Google) trên cùng một bảng.

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính, tự tăng
name	VARCHAR(100)	NULL — họ tên hiển thị
phone	VARCHAR(15)	UNIQUE, NULL — dùng đăng nhập/đăng ký bằng OTP
phone_verified_at	TIMESTAMP	NULL — thời điểm xác thực OTP thành công lần đầu
email	VARCHAR(100)	UNIQUE, NULL
username	VARCHAR(50)	UNIQUE, NULL
password_hash	VARCHAR(255)	NULL — băm BCrypt
has_password	BOOLEAN	NOT NULL, DEFAULT FALSE — cờ phân biệt tài khoản đã đặt mật khẩu hay chỉ đăng nhập OTP/Google
avatar_url	VARCHAR(500)	NULL
role	ENUM	'CUSTOMER','STAFF','ADMIN' — NOT NULL, DEFAULT 'CUSTOMER'
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE
created_at / updated_at	TIMESTAMP	Tự động gán / cập nhật

otp_verifications

Lịch sử mã OTP đã gửi. Không có khoá ngoại tới users (áp dụng cho cả luồng REGISTER khi tài khoản chưa tồn tại).

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
phone	VARCHAR(15)	NOT NULL
otp_code	VARCHAR(6)	NOT NULL
type	ENUM	'REGISTER','LOGIN','RESET_PASSWORD' — NOT NULL
is_used	BOOLEAN	NOT NULL, DEFAULT FALSE

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
expires_at	TIMESTAMP	NOT NULL
created_at	TIMESTAMP	—
(index)	—	INDEX idx_otp_phone(phone); INDEX idx_otp_expires(expires_at)

social_accounts

Liên kết tài khoản đăng nhập mạng xã hội (hiện chỉ Google).

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
user_id	BIGINT	NOT NULL — FK → users(id) ON DELETE CASCADE
provider	ENUM	'GOOGLE' — NOT NULL (thiết kế cho phép mở rộng provider khác nhưng hiện chỉ có Google)
provider_id	VARCHAR(255)	NOT NULL
email	VARCHAR(100)	NULL
created_at	TIMESTAMP	—
(unique)	—	UNIQUE(provider, provider_id)

4.2 Nhóm Địa chỉ (1 bảng)

addresses

Sổ địa chỉ giao hàng của người dùng (nhiều địa chỉ/1 user).

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
user_id	BIGINT	NOT NULL — FK → users(id) ON DELETE CASCADE
full_name, phone	VARCHAR	NOT NULL — snapshot riêng, độc lập với users.name/users.phone
province, district, ward	VARCHAR(100)	NOT NULL — nhãn địa danh dạng text
detail	TEXT	NOT NULL
is_default	BOOLEAN	NOT NULL, DEFAULT FALSE
district_id	INT	NULL — mã district của GHN
ward_code	VARCHAR(10)	NULL — mã ward của GHN; khi NULL, phí ship dùng mức cố định 30.000đ (xem Mục 7 — GHN)

4.3 Nhóm Sản phẩm (8 bảng)

categories

Danh mục sản phẩm.

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
name	VARCHAR(100)	NOT NULL
slug	VARCHAR(100)	NOT NULL, UNIQUE
description	TEXT	NULL
image_url	VARCHAR(500)	NULL
sort_order	INT	NOT NULL, DEFAULT 0
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE

flavors

Danh mục hương vị (thuộc tính biến thể sản phẩm).

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
name	VARCHAR(100)	NOT NULL
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE

sizes

Danh mục kích cỡ (thuộc tính biến thể sản phẩm).

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
name	VARCHAR(50)	NOT NULL
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE

products

Sản phẩm gốc (chưa gồm biến thể giá/tồn kho — xem product_variants).

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
category_id	BIGINT	NOT NULL — FK → categories(id)
name	VARCHAR(200)	NOT NULL
slug	VARCHAR(200)	NOT NULL, UNIQUE
description	TEXT	NULL
avg_rating	FLOAT	NOT NULL, DEFAULT 0 — số liệu tổng hợp (denormalized) từ reviews
review_count	INT	NOT NULL, DEFAULT 0 — số liệu tổng hợp (denormalized) từ reviews
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
is_deleted	BOOLEAN	NOT NULL, DEFAULT FALSE — thêm bằng ALTER TABLE; xoá mềm (soft-delete); có INDEX riêng
created_at / updated_at	TIMESTAMP	Tự động gán / cập nhật

avg_rating và review_count là dữ liệu tổng hợp (denormalized) — không tự động đúng theo ràng buộc CSDL mà phụ thuộc mã nguồn cập nhật đúng thời điểm (khi review được duyệt/sửa/xoá). Đây là điểm cần kiểm thử hồi quy: tạo/sửa/xoá đánh giá rồi đối chiếu lại 2 giá trị này trên trang sản phẩm.

product_images

Ảnh sản phẩm (1 sản phẩm có nhiều ảnh).

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
product_id	BIGINT	NOT NULL — FK → products(id) ON DELETE CASCADE
image_url	VARCHAR(500)	NOT NULL — URL Cloudinary (xem Mục 7)
is_primary	BOOLEAN	NOT NULL, DEFAULT FALSE
sort_order	INT	NOT NULL, DEFAULT 0

product_tags

Gán nhãn tự do cho sản phẩm (bestseller, organic, v.v.).

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
product_id	BIGINT	NOT NULL — FK → products(id) ON DELETE CASCADE
tag	VARCHAR(50)	NOT NULL — chuỗi tự do, KHÔNG có khoá ngoại tới tag_definitions.name

⚠ product_tags.tag là văn bản tự do, không ràng buộc khoá ngoại tới tag_definitions — CSDL không ngăn việc gán một tag không tồn tại trong danh mục chuẩn (tag_definitions). Cần kiểm thử: tạo/sửa sản phẩm với tag tùy ý ở tầng API xem hệ thống có validate ở tầng ứng dụng hay không.

tag_definitions

Danh mục nhãn chuẩn (dữ liệu khởi tạo sẵn 4 dòng: bestseller, organic, new, sugar-free).

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
name	VARCHAR(50)	NOT NULL, UNIQUE
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE
sort_order	INT	NOT NULL, DEFAULT 0

product_variants

Biến thể sản phẩm (hương vị × kích cỡ) — nơi lưu giá bán và tồn kho thực tế.

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
product_id	BIGINT	NOT NULL — FK → products(id) ON DELETE CASCADE
flavor_id	BIGINT	NULL — FK → flavors(id)
size_id	BIGINT	NULL — FK → sizes(id)
original_price	DECIMAL(12,2)	NOT NULL
sale_price	DECIMAL(12,2)	NOT NULL
discount_percent	DECIMAL(5,2)	NOT NULL, DEFAULT 0
stock_qty	INT	NOT NULL, DEFAULT 0
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE
sort_order	INT	NOT NULL, DEFAULT 0
weight_gram	INT	NOT NULL, DEFAULT 500 — trọng lượng (gram), dùng tính phí GHN theo cân nặng

flavor_id và size_id đều NULL-able — CSDL cho phép một biến thể không gắn hương vị lẫn kích cỡ nào. Cần kiểm thử xác nhận ứng dụng có chặn tạo biến thể "rỗng" như vậy ở tầng validate hay không.

4.4 Nhóm Giỏ hàng (2 bảng)

carts

Giỏ hàng — quan hệ 1-1 với user (ràng buộc UNIQUE trên user_id).

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
user_id	BIGINT	NOT NULL, UNIQUE — FK → users(id) ON DELETE CASCADE
created_at / updated_at	TIMESTAMP	Tự động gán / cập nhật

cart_items

Dòng sản phẩm trong giỏ hàng.

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
cart_id	BIGINT	NOT NULL — FK → carts(id) ON DELETE CASCADE
product_id	BIGINT	NOT NULL — FK → products(id) (lưu kèm để truy vấn thuận tiện, có thể suy ra từ variant_id)
variant_id	BIGINT	NOT NULL — FK → product_variants(id)
quantity	INT	NOT NULL, DEFAULT 1
updated_at	TIMESTAMP	Tự động cập nhật
(unique/index)	—	UNIQUE(cart_id, variant_id); INDEX(product_id)

4.5 Nhóm Đơn hàng & Thanh toán (3 bảng)

orders

Đơn hàng — bảng trung tâm của toàn hệ thống.

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
user_id	BIGINT	NOT NULL — FK → users(id)
promotion_id	BIGINT	NULL — FK → promotions(id) ON DELETE SET NULL
order_code	VARCHAR(50)	NOT NULL, UNIQUE — định dạng GJH-XXXXXXX
subtotal, discount_amount, shipping_fee, total_amount	DECIMAL(12,2)	NOT NULL (discount_amount, shipping_fee DEFAULT 0)
status	ENUM	'PENDING','CONFIRMED','SHIPPING','DELIVERED','CANCELLED' — DEFAULT 'PENDING'
payment_status	ENUM	'PENDING','PAID','REFUND_PENDING','REFUNDED' — DEFAULT 'PENDING'
shipping_address	JSON	NOT NULL — snapshot địa chỉ tại thời điểm đặt hàng, KHÔNG phải khoá ngoại tới addresses
note	TEXT	NULL
cancel_reason	VARCHAR(500)	NULL
cancelled_by	VARCHAR(20)	NULL — 'CUSTOMER' hoặc 'SYSTEM' theo quy ước ứng dụng (cột thô, không phải ENUM ở tầng CSDL)
ghn_order_code	VARCHAR(50)	NULL — mã vận đơn GHN, thêm bằng ALTER TABLE
expires_at	DATETIME	NULL — hạn thanh toán cho đơn online (QR/chuyển khoản); NULL với COD
created_at / updated_at	TIMESTAMP	Tự động gán / cập nhật
(index)	—	INDEX(status); INDEX(user_id)

shipping_address là cột JSON lưu "ảnh chụp" (snapshot) địa chỉ ngay lúc đặt hàng, không tham chiếu khoá ngoại tới bảng addresses — sửa/xoá địa chỉ trong số địa chỉ sau này không ảnh hưởng đơn hàng đã đặt. cancelled_by là VARCHAR thô nên CSDL không chặn giá trị ngoài 'CUSTOMER'/'SYSTEM' — chỉ được kiểm soát ở tầng Java enum.

order_items

Dòng sản phẩm trong đơn hàng — snapshot tên/giá tại thời điểm đặt.

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
order_id	BIGINT	NOT NULL — FK → orders(id) ON DELETE CASCADE
product_id	BIGINT	NOT NULL — FK → products(id)
variant_id	BIGINT	NOT NULL — FK → product_variants(id)
product_name, variant_name	VARCHAR	NOT NULL — snapshot tên tại thời điểm đặt hàng

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
unit_price	DECIMAL(12,2)	NOT NULL — snapshot giá tại thời điểm đặt hàng
quantity	INT	NOT NULL
subtotal	DECIMAL(12,2)	NOT NULL
(index)	—	INDEX(order_id); INDEX(product_id)

Việc snapshot `product_name/variant_name/unit_price` khớp với thiết kế xoá mềm (`products.is_deleted`): lịch sử đơn hàng không bị vỡ dữ liệu kể cả khi sản phẩm gốc đã bị xoá hoặc đổi giá sau này.

payments

Giao dịch thanh toán của đơn hàng (1 đơn có thể có nhiều bản ghi payment theo thời gian).

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
order_id	BIGINT	NOT NULL — FK → orders(id) ON DELETE CASCADE
method	ENUM	'COD','VNPAY','MOMO','BANK_TRANSFER' — NOT NULL
status	ENUM	'PENDING','SUCCESS','FAILED','REFUNDED' — DEFAULT 'PENDING'
amount	DECIMAL(12,2)	NOT NULL
transaction_id	VARCHAR(255)	NULL
paid_at	TIMESTAMP	NULL
note	TEXT	NULL
created_at	TIMESTAMP	—
(index)	—	INDEX(order_id)

Ở tầng CSDL, `MOMO` và `BANK_TRANSFER` là 2 giá trị enum tách biệt — thiết kế dữ liệu vẫn phân biệt 2 kênh thanh toán này, dù hiện tại cả hai đang dùng chung một cơ chế xử lý mô phỏng ở tầng backend (xem Mục 7 và Mục 11).

4.6 Nhóm Khuyến mãi (2 bảng)

promotions

Mã khuyến mãi — áp dụng công khai hoặc gán riêng cho 1 người dùng.

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
code	VARCHAR(50)	NOT NULL, UNIQUE
name	VARCHAR(200)	NOT NULL
type	ENUM	'PERCENT','FIXED' — NOT NULL
value	DECIMAL(12,2)	NOT NULL — 1 cột dùng chung: là % nếu type=PERCENT, là số tiền VNĐ nếu type=FIXED
min_order_value	DECIMAL(12,2)	NOT NULL, DEFAULT 0
free_shipping	BOOLEAN	NOT NULL, DEFAULT FALSE

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
target	ENUM	'PUBLIC','PERSONAL' — DEFAULT 'PUBLIC'
user_id	BIGINT	NULL — FK → users(id) ON DELETE SET NULL; chỉ có giá trị khi target='PERSONAL'
max_uses	INT	NULL — NULL nghĩa là không giới hạn tổng lượt dùng
max_uses_per_user	INT	NULL
used_count	INT	NOT NULL, DEFAULT 0
starts_at / ends_at	DATETIME	NOT NULL
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE

Cột value dùng chung cho cả 2 loại giảm giá (% hoặc VNĐ cố định) tùy theo type — cần kiểm thử validate: giá trị PERCENT có bị chặn trên 100 hay không, và FIXED có được chặn âm/vượt quá tổng đơn hay không.

promotion_usages

Nhật ký sử dụng mã khuyến mãi — dùng để tính max_uses_per_user.

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
promotion_id	BIGINT	NOT NULL — FK → promotions(id)
user_id	BIGINT	NOT NULL — FK → users(id)
order_id	BIGINT	NOT NULL — FK → orders(id)
used_at	TIMESTAMP	—
(index)	—	INDEX(promotion_id, user_id); INDEX(order_id)

4.7 Nhóm Đánh giá (1 bảng)

reviews

Đánh giá sản phẩm — bắt buộc gắn với 1 đơn hàng cụ thể.

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
product_id	BIGINT	NOT NULL — FK → products(id)
user_id	BIGINT	NOT NULL — FK → users(id)
order_id	BIGINT	NOT NULL — FK → orders(id)
rating	TINYINT	NOT NULL — CHECK (rating BETWEEN 1 AND 5)
comment	TEXT	NULL
image_url	VARCHAR(500)	NULL
is_approved	BOOLEAN	NOT NULL, DEFAULT TRUE — đánh giá hiển thị công khai ngay khi tạo, không qua duyệt trước
reply	TEXT	NULL — phản hồi từ Admin/Staff

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
replied_at	DATETIME	NULL
product_name	VARCHAR(200)	NULL — snapshot tên sản phẩm lúc đánh giá
created_at	TIMESTAMP	—
(unique/index)	—	UNIQUE(product_id, user_id, order_id); INDEX(product_id)

order_id NOT NULL kết hợp UNIQUE(product_id, user_id, order_id) cho thấy đánh giá bắt buộc gắn với một đơn hàng cụ thể — một khách hàng có thể đánh giá lại cùng sản phẩm nếu mua ở đơn khác, nhưng không thể đánh giá trùng cho cùng 1 đơn. Cần kiểm thử xác nhận điều kiện chính xác để được đánh giá (ví dụ: đơn phải ở trạng thái DELIVERED hay chỉ cần tồn tại order_items khớp sản phẩm).

4.8 Nhóm Khác (3 bảng)

contacts

Form liên hệ công khai — không gắn với tài khoản đăng nhập.

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
full_name, email	VARCHAR(100)	NOT NULL
phone	VARCHAR(15)	NULL
subject	VARCHAR(200)	NOT NULL
message	TEXT	NOT NULL
status	ENUM	'NEW','IN_PROGRESS','RESOLVED' — DEFAULT 'NEW'
reply	TEXT	NULL
replied_at	DATETIME	NULL
replied_by_name	VARCHAR(100)	NULL
created_at	TIMESTAMP	—

shipping_policies

Nội dung các trang chính sách tỉnh (vận chuyển, đổi trả, bảo hành, điều khoản) — mỗi loại 1 dòng duy nhất.

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
type	ENUM	'SHIPPING','RETURN','WARRANTY','TERMS' — NOT NULL, UNIQUE
title	VARCHAR(200)	NOT NULL
content	LONGTEXT	NOT NULL — nội dung HTML từ trình soạn thảo Tiptap
sort_order	INT	NOT NULL, DEFAULT 0
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE
updated_at	TIMESTAMP	Tự động cập nhật

banners

Banner quảng cáo hiển thị trang chủ.

Cột	Kiểu dữ liệu	Ràng buộc / Ghi chú
id	BIGINT	Khoá chính
title	VARCHAR(200)	NOT NULL
description	VARCHAR(500)	NULL
image_url	VARCHAR(500)	NOT NULL
link_url	VARCHAR(500)	NULL
sort_order	INT	NOT NULL, DEFAULT 0
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE

5. Kiến trúc Backend

5.1 Cấu trúc package

Mã nguồn backend tổ chức theo package Java `com.greenjuicehub.backend`, tách lớp theo vai trò kỹ thuật (kiến trúc dạng layered, không theo module/feature):

- **config/** và **config/properties/** — AppConfig, CorsConfig, RedisConfig, SecurityConfig, và 6 lớp *Properties (GeminiProperties, GhnProperties, JwtProperties, OtpProperties, SePayProperties, VnpayProperties) ánh xạ trực tiếp cấu hình từ application.yml bằng @ConfigurationProperties.
- **controller/** — 26 lớp @RestController, tổng cộng 124 endpoint (chi tiết đầy đủ tại Phụ lục A của Tài liệu SRS).
- **dto/<domain>/request|response/** — khoảng 75 lớp DTO, tách rõ request/response theo từng domain nghiệp vụ (auth, adminProduct, adminOrder, cart, order, review, v.v.), không tái sử dụng entity làm dữ liệu trả về API.
- **entity/** — 23 lớp entity JPA, tương ứng 1-1 với 23 bảng CSDL (Mục 4).
- **exception/** — ApplicationException (exception nghiệp vụ tự định nghĩa, mang theo HTTP status) và GlobalExceptionHandler (xem Mục 5.3).
- **mapper/** — 10 lớp mapper thủ công chuyển đổi entity ↔ DTO (viết tay, không dùng thư viện MapStruct/ModelMapper).
- **middleware/** — JwtAuthFilter, bộ lọc xác thực JWT duy nhất của toàn hệ thống (xem Mục 5.2).
- **repository/** — 23 interface Spring Data JPA, tương ứng 1-1 với entity; một số dùng JPA Specification để lọc động (ví dụ danh sách sản phẩm theo nhiều tiêu chí).
- **service/<domain>/** (+ impl/) — tách interface (I*Service) và lớp triển khai theo 16 domain: auth, banner, cart, contact, dashboard, email, order, payment, policy, product, promotion, recommendation, review, sepay, shipping, user.
- **utils/** — JwtUtil (sinh/giải mã/kiểm tra JWT).

5.2 Luồng xác thực & phân quyền (JWT)

SecurityConfig cấu hình: tắt CSRF (API thuần JSON, không dùng session/cookie cho xác thực), bật CORS qua một bean CorsConfigurationSource riêng, chính sách phiên STATELESS (không lưu session phía server), gán JwtAuthFilter vào chuỗi filter ngay trước UsernamePasswordAuthenticationFilter, và dùng BCryptPasswordEncoder để băm mật khẩu. Danh sách endpoint permitAll() được khai báo tường minh (19 pattern, gồm các nhóm auth công khai, sản phẩm, banner, chính sách, vận chuyển, webhook, và các endpoint VNPay IPN/Return) — toàn bộ endpoint còn lại bắt buộc phải có xác thực (anyRequest().authenticated()).

Quan trọng: SecurityConfig chỉ đảm bảo request "đã đăng nhập" (authenticated), KHÔNG kiểm tra đúng vai trò (role). Việc giới hạn theo vai trò CUSTOMER/STAFF/ADMIN nằm hoàn toàn ở annotation @PreAuthorize gắn riêng trên từng controller/method. Đây chính là lý do AdminTagController (không có @PreAuthorize — xem Mục 11) vẫn bị chặn bởi "phải đăng nhập" nhưng KHÔNG bị chặn theo vai trò: bất kỳ tài khoản nào đã đăng nhập, kể cả CUSTOMER thường, cũng gọi được các endpoint quản lý tag.

Luồng xử lý của JwtAuthFilter (mỗi request tới endpoint không nằm trong danh sách bỏ qua):

Bước 1: Nếu đường dẫn request là /api/auth/refresh hoặc /api/auth/logout → bỏ qua hoàn toàn bộ lọc (shouldNotFilter), request đi thẳng vào controller.

Bước 2: Nếu không có header Authorization dạng "Bearer ..." → cho đi tiếp mà không gán SecurityContext (endpoint public tự xử lý bình thường; endpoint cần xác thực sẽ tự bị Spring Security chặn ở bước anyRequest().authenticated()).

Bước 3: Kiểm tra token hợp lệ và còn hạn (jwtUtil.isTokenValid — gồm luôn bước kiểm tra hết hạn trong cùng 1 lần parse) → sai thì trả 401 ngay.

Bước 4: Kiểm tra token có nằm trong danh sách đen (Redis, TokenBlacklistService — gán khi người dùng logout) hay không → có thì trả 401.

Bước 5: Kiểm tra claim "type" của token phải đúng là "access" → chặn trường hợp dùng nhầm refresh token làm access token, trả 401 nếu sai.

Bước 6: Trích userId và role từ token; nếu một trong hai null → trả 401 (tránh tạo authority "ROLE_NULL").

Bước 7: Gán Authentication vào SecurityContextHolder với authority "ROLE_" + role viết hoa (chỉ gán nếu context chưa có sẵn authentication, tránh ghi đè).

Bước 8: Bất kỳ exception nào phát sinh trong toàn bộ quá trình trên đều được bắt gọn (try/catch bao ngoài) → xoá SecurityContext, trả 401, không để lộ stack trace ra response.

Mã nguồn JwtAuthFilter có các chú thích tiếng Việt đánh số "FIX 1" đến "FIX 6" ngay trong thân hàm, cho thấy bộ lọc đã được tinh chỉnh qua nhiều lần sửa lỗi thực tế. Nhóm kiểm thử nên ưu tiên viết test case đúng vào các nhánh lỗi này: token rỗng/sai định dạng, token hết hạn, token đã bị thu hồi (đã logout), token đúng nhưng sai loại (dùng refresh token thay access token), và token hợp lệ nhưng thiếu claim userId/role.

5.3 Xử lý ngoại lệ tập trung

GlobalExceptionHandler (@RestControllerAdvice) xử lý tập trung 3 nhóm lỗi, luôn trả về cùng một khuôn dạng JSON {status, message, timestamp}:

Loại exception	Xử lý
AppException (tự định nghĩa)	Trả đúng HTTP status và message do tầng service/ng nghiệp vụ chủ động ném ra
MethodArgumentNotValidException (lỗi @Valid trên DTO)	Trả 400, message lấy từ lỗi field đầu tiên tìm thấy (dạng "tênField: thông báo lỗi")
Exception (bắt tất cả các lỗi còn lại)	Ghi log đầy đủ kèm stack trace ở phía server (log.error), nhưng chỉ trả về client thông báo chung "Lỗi hệ thống" kèm mã 500 — không lộ chi tiết kỹ thuật

GlobalExceptionHandler không có @ExceptionHandler riêng cho AccessDeniedException (lỗi phát sinh khi @PreAuthorize chặn một request đã đăng nhập nhưng sai vai trò). Cần kiểm thử xác nhận thực tế: phản hồi 403 trong trường hợp này có theo cùng khuôn dạng {status, message, timestamp} như các lỗi khác hay không, vì cơ chế xử lý mặc định của Spring Security cho AccessDeniedException hoạt động độc lập với @RestControllerAdvice.

5.4 Tác vụ nền theo lịch (Scheduled Jobs)

Hệ thống có 2 tác vụ nền chạy tự động bằng @Scheduled (được bật ở lớp cấu hình gốc), cùng nằm trong package service/order:

Lớp	Lịch chạy	Điều kiện quét	Hành động
AutoCancelScheduler	fixedRate = 5 phút	orders.status = PENDING và expires_at < thời điểm hiện tại	Hoàn kho + hoàn/giải phóng lượt dùng khuyến mãi (restockAndRefundPromo); chuyển status → CANCELLED, cancelled_by = SYSTEM, gán cancel_reason cố định. Có kiểm tra lại payment_status trước khi huỷ để bỏ qua đơn

Lớp	Lịch chạy	Điều kiện quét	Hành động
			vừa thanh toán xong (tránh race condition).
AutoDeliverScheduler	cron "0 0 2 * * *" (2:00 sáng mỗi ngày)	orders.status = SHIPPING và updated_at cách hiện tại > 7 ngày	Chuyển status → DELIVERED. Nếu phương thức thanh toán là COD và chưa ở trạng thái SUCCESS thì tự động đánh dấu payment = SUCCESS, paid_at = hiện tại, order.payment_status = PAID.

AutoDeliverScheduler suy đoán đơn hàng "đã giao thành công" hoàn toàn dựa trên mốc thời gian (7 ngày kể từ lần cập nhật cuối), không có bước xác nhận thực tế từ đơn vị vận chuyển GHN. Đây là điểm cần kiểm thử biên quan trọng: hành vi đúng ngay tại mốc 7 ngày, và rủi ro nghiệp vụ nếu đơn thực tế chưa giao xong nhưng đã bị hệ thống tự đánh dấu DELIVERED (và tự PAID nếu COD).

5.5 Cấu hình & khoá bí mật

- Cấu hình được tách theo Spring Profile chuẩn (application.yml cho cấu hình chung, application-local.yml cho giá trị bí mật của môi trường local) — tài liệu này không mở và không trích dẫn giá trị bí mật (khoá JWT, mật khẩu ứng dụng Gmail, secret reCAPTCHA, token GHN, hash secret VNPay, khoá SePay).
- 6 lớp *Properties (Mục 5.1) dùng @ConfigurationProperties để ánh xạ cấu hình theo kiểu type-safe thay vì đọc trực tiếp bằng @Value rải rác — thuận lợi cho việc mock/kiểm thử cấu hình.
- RedisConfig thiết lập kết nối Redis dùng chung cho 3 mục đích độc lập: khoá đăng nhập OTP sai/quá số lần gửi (OtpLockService), khoá đăng nhập sai mật khẩu quá số lần (PasswordAttemptService), và danh sách token bị thu hồi khi logout (TokenBlacklistService) — cả 3 đều tận dụng cơ chế TTL tự hết hạn của Redis thay vì phải tự dọn dữ liệu.

6. Kiến trúc Frontend

6.1 Cấu trúc thư mục

- **api/** — 21 file: 20 module gọi REST API tách theo domain (authApi, productApi, cartApi, orderApi, paymentApi, adminProductApi, v.v.) và axiosConfig.js — cấu hình instance Axios dùng chung cho toàn app (xem Mục 6.4).
- **components/** — chia theo domain: cart/, common/ (ProtectedRoute, Preloader, RichText/RichTextEditor, ThemeToggle), layout/ (Header, Footer, Nav, AdminLayout, AdminSidebar, AdminTopbar), order/ (BankTransferModal, MomoTransferModal, OrderSuccessModal), product/ (FilterSidebar, ImageGallery, ProductCard, ReviewForm, ReviewList...), user/ (LocationSelect, ProfileModal).
- **constants/** — assets.js (đường dẫn tài nguyên tĩnh dùng chung).
- **hooks/** — 7 custom hook: useAdminRole, useCartAnimation, useDrawerTransition, usePageReady, useProductDetail, useProducts (lọc/debounce/lưu trạng thái lọc theo phiên), useTheme.
- **pages/** — chia theo domain: admin/ (dashboard, banners, contact, order, policy, product, promotion, reviews, user), ai-consultant/, auth/ (7 trang), checkout/, contact/, home/, orders/, payment/ (VnpayResultPage), policies/, products/, settings/.
- **store/** — 5 store Zustand (xem Mục 6.3).
- **utils/** — sharedObserver.js (IntersectionObserver dùng chung cho hiệu ứng cuộn/lazy-load).

6.2 Định tuyến (React Router 7)

Toàn bộ route khai báo tập trung tại App.jsx, chia 4 nhóm theo mức độ bảo vệ:

Nhóm	Đường dẫn	Component	Bảo vệ
Chỉ khách (GuestRoute)	/login	LoginPage	Chặn nếu đã đăng nhập
Chỉ khách (GuestRoute)	/login-password	LoginPasswordPage	Chặn nếu đã đăng nhập
Chỉ khách (GuestRoute)	/verify-otp	VerifyOtpPage	Chặn nếu đã đăng nhập
Chỉ khách (GuestRoute)	/set-password	SetPasswordPage	Chặn nếu đã đăng nhập
Chỉ khách (GuestRoute)	/forgot-password	ForgotPasswordPage	Chặn nếu đã đăng nhập
Chỉ khách (GuestRoute)	/reset-password	ResetPasswordPage	Chặn nếu đã đăng nhập
Chỉ khách (GuestRoute)	/login-option	LoginOptionPage	Chặn nếu đã đăng nhập
Công khai	/	HomePage	Không
Công khai	/products	ProductsPage	Không
Công khai	/products/:slug	ProductDetailPage	Không
Công khai	/contact	ContactPage	Không
Công khai	/policies/:type	PolicyPage	Không

Nhóm	Đường dẫn	Component	Bảo vệ
Công khai	/payment/vnpay/result	VnpayResultPage (không bọc MainLayout)	Không
Công khai	/ai-tu-van	AiConsultantPage	Không
Customer	/checkout	CheckoutPage	ProtectedRoute — allowedRoles=['CUSTOMER']
Customer	/orders	OrdersListPage	ProtectedRoute — allowedRoles=['CUSTOMER']
Customer	/orders/:orderId	OrderDetailPage	ProtectedRoute — allowedRoles=['CUSTOMER']
Customer	/settings	SettingsPage	ProtectedRoute — allowedRoles=['CUSTOMER']
Admin + Staff	/admin (index)	AdminDashboardPage	ProtectedRoute — allowedRoles=['ADMIN','STAFF']
Admin + Staff	/admin/users	AdminUsersPage	(lồng trong AdminLayout, cùng bảo vệ trên)
Admin + Staff	/admin/orders	AdminOrdersPage	(lồng trong AdminLayout, cùng bảo vệ trên)
Admin + Staff	/admin/orders/:orderId	AdminOrderDetailPage	(lồng trong AdminLayout, cùng bảo vệ trên)
Admin + Staff	/admin/products	AdminProductsPage	(lồng trong AdminLayout, cùng bảo vệ trên)
Admin + Staff	/admin/products/new	AdminProductFormPage	(lồng trong AdminLayout, cùng bảo vệ trên)
Admin + Staff	/admin/products/:id/edit	AdminProductFormPage	(lồng trong AdminLayout, cùng bảo vệ trên)
Admin + Staff	/admin/promotions	AdminPromotionsPage	(lồng trong AdminLayout, cùng bảo vệ trên)
Admin + Staff	/admin/reviews	AdminReviewsPage	(lồng trong AdminLayout, cùng bảo vệ trên)
Admin + Staff	/admin/contacts	AdminContactsPage	(lồng trong AdminLayout, cùng bảo vệ trên)
Admin + Staff	/admin/banners	AdminBannersPage	(lồng trong AdminLayout, cùng bảo vệ trên)
Admin + Staff	/admin/policies	AdminPoliciesPage	(lồng trong AdminLayout, cùng bảo vệ trên)
Dự phòng	/404 và *	NotFoundPage	Không

ProtectedRoute của toàn bộ khu vực /admin chấp nhận CẢ HAI vai trò ADMIN và STAFF ở tầng định tuyến frontend — việc giới hạn STAFF chỉ được xem chứ không được sửa (ví dụ theo Phụ lục A của SRS: AdminProductController cho STAFF "chỉ xem", ADMIN "toàn quyền") phải được kiểm chứng lại ở tầng giao diện (ẩn/khoá nút thao tác) và tầng API (@PreAuthorize), KHÔNG được đảm bảo bởi route. Đây là điểm kiểm thử giao diện đáng chú ý cho vai trò STAFF.

6.3 Quản lý trạng thái (Zustand)

- **authStore.js** — trạng thái đăng nhập (isLoggedIn), thông tin người dùng hiện tại, accessToken/refreshToken (đồng bộ hai chiều với localStorage thông qua axiosConfig.js); action

fetchMe() được App.jsx gọi đúng 1 lần khi ứng dụng khởi động nếu người dùng đã đăng nhập từ trước.

- **useCartStore.js** — danh sách sản phẩm trong giỏ hàng, fetchCart(); resetCart() được gọi khi đăng xuất (forceLogout trong axiosConfig.js).
- **useOrderStore.js** — trạng thái liên quan luồng đặt hàng/thanh toán khi ở trang Checkout.
- **useAppStore.js** — chờ pageReady, dùng để quyết định thời điểm tắt Preloader (màn hình chờ) khi vào trang chủ.
- **useProfileModalStore.js** — trạng thái đóng/mở modal thông tin tài khoản (ProfileModal).

6.4 Lớp gọi API & phiên đăng nhập (axiosConfig.js)

Toàn bộ request API đi qua một instance Axios dùng chung (base URL lấy từ biến môi trường VITE_API_URL), với 2 interceptor xử lý các tình huống đặc thù của hệ thống:

- Request interceptor: tự động gắn header Authorization: Bearer <accessToken> (đọc từ localStorage) vào mọi request chưa có sẵn header này, trừ khi request đánh dấu skipAccessToken.
- Response interceptor — xử lý lỗi mạng do "cold start": nếu request lỗi mà không nhận được response (mã ERR_NETWORK/ECONNABORTED hoặc không xác định) và chưa từng thử lại, hệ thống đợi đúng 12.000ms (hằng số COLD_START_RETRY_DELAY_MS — do backend Railway "ngủ" khi không có traffic) rồi tự động gọi lại request đúng 1 lần; không áp dụng cơ chế này cho chính các request tới /api/auth/**.
- Response interceptor — xử lý 401: nếu không có refreshToken → đăng xuất ngay; nếu đang có một lượt làm mới token khác chạy dở → request hiện tại được xếp vào hàng đợi, chờ lượt đó xong rồi tự gắn token mới và gọi lại (tránh gọi /api/auth/refresh nhiều lần đồng thời); nếu chưa có lượt nào → gọi POST /api/auth/refresh (Bearer = refreshToken, timeout riêng 15.000ms để chịu được cold start), lưu accessToken/refreshToken mới vào localStorage và cập nhật authStore, rồi phát lại request gốc cùng toàn bộ hàng đợi.
- Nếu chính request làm mới token trả về 401/403 (refresh token thật sự hết hạn/không hợp lệ) → đăng xuất. Nếu request đó lỗi vì lý do khác (mạng chập chờn, server đang cold start) → KHÔNG đăng xuất, chỉ đặt lại cờ để có thể thử lại sau, tránh đăng xuất oan người dùng.
- forceLogout(): xoá accessToken/refreshToken/role khỏi localStorage, reset store giỏ hàng, rồi điều hướng cứng (window.location.href, tải lại toàn trang) về /login.

accessToken và refreshToken được lưu trong localStorage (không phải cookie HttpOnly). Đây là đặc điểm kiến trúc — không phải lỗi cấu hình — nhưng cần lưu ý khi kiểm thử bảo mật vì localStorage có thể bị đọc bởi bất kỳ đoạn JavaScript nào chạy trên trang (bề mặt tấn công XSS lý thuyết cao hơn so với cookie HttpOnly). Xem thêm Mục 9 và Mục 11.

6.5 Trang & thành phần chính

- Luồng thanh toán tại CheckoutPage dùng 3 modal riêng theo phương thức: BankTransferModal và MomoTransferModal (hiển thị mã VietQR động + polling trạng thái mỗi 3 giây), và OrderSuccessModal (xác nhận đặt hàng thành công) — xem chi tiết luồng ở Mục 8.5.
- Upload ảnh (avatar, ảnh sản phẩm, ảnh banner, ảnh đính kèm đánh giá) đều dùng chung cơ chế uploadImage() trong cloudinaryApi.js: trình duyệt gửi file trực tiếp lên Cloudinary bằng "unsigned upload preset" (preset riêng cho products/reviews, banners dùng chung preset với products), KHÔNG đi qua backend — backend chỉ nhận và lưu lại secure_url trả về sau khi upload xong. Đây là điểm quan trọng khi kiểm thử: lỗi upload ảnh xảy ra ở Cloudinary, không nằm trong log backend.
- SettingsPage: chức năng xoá tài khoản và tùy chọn nhận thông báo hiện là placeholder ở giao diện — chưa gọi API backend tương ứng (xem Mục 11).

- AdminDashboardPage: 6 chỉ số tổng quan (dạng thẻ số liệu) và 1 biểu đồ doanh thu (thư viện Recharts).
- useProducts.js: hook trung tâm cho trang danh sách sản phẩm — quản lý bộ lọc (danh mục, giá, tag...), debounce ô tìm kiếm, và lưu lại trạng thái lọc theo phiên làm việc (session) để giữ nguyên khi quay lại trang.

7. Tích hợp bên thứ ba

Hệ thống tích hợp 7 dịch vụ bên ngoài. Đây đều là các điểm phụ thuộc mạng bên ngoài — cần có kịch bản kiểm thử riêng cho trường hợp dịch vụ bên thứ ba chậm/lỗi/không phản hồi.

Dịch vụ	Mục đích	Cơ chế chính
Google OAuth2	Đăng nhập bằng tài khoản Google	Frontend (@react-oauth/google) lấy idToken → gửi POST /api/auth/google → backend xác minh bằng Google API Client (GoogleTokenVerifier) → tự tạo social_accounts + users nếu là lần đầu
Google Gemini	Chatbot tư vấn/gợi ý sản phẩm bằng AI	RecommendationController /api/recommendation (công khai) → RecommendationPromptBuilder dựng prompt từ câu hỏi + danh sách sản phẩm → GeminiRecommendationServiceImpl gọi Gemini API
VNPay	Cổng thanh toán trực tuyến thật	URL thanh toán ký HMAC-SHA512 (VnpayServiceImpl); callback qua 2 kênh permitAll độc lập: IPN (server-to-server, xác nhận chính thức) và Return (trình duyệt, chỉ hiển thị kết quả)
GHN (Giao Hàng Nhanh)	Tra cứu địa chỉ & tính phí vận chuyển	Tra cứu tỉnh/huyện/xã theo tầng; tính phí thời gian thực theo weight_gram (GhnService); hiện chỉ hỗ trợ nội thành Hồ Chí Minh (giới hạn ở frontend)
SePay	Xác nhận giao dịch chuyển khoản ngân hàng	Webhook /api/webhooks, xác thực bằng header Authorization: apikey {key} riêng (không dùng JWT); khớp mã đơn hàng bằng regex GJH-XXXXXXX; luôn trả HTTP 200
Cloudinary	Lưu trữ ảnh (sản phẩm, banner, đánh giá, avatar)	Trình duyệt upload trực tiếp bằng unsigned preset (cloudinaryApi.js); backend chỉ nhận và lưu secure_url, không xử lý file ảnh
Gmail SMTP	Gửi email (phản hồi liên hệ, thông báo)	Spring Boot Starter Mail + Thymeleaf (EmailService) dựng email HTML, gửi qua máy chủ SMTP Gmail

⚠ Trong bảng payments, MOMO là 1 giá trị enum riêng biệt với BANK_TRANSFER, nhưng KHÔNG có tích hợp API MoMo thật: cả hai phương thức hiện dùng chung một cơ chế xử lý mô phỏng (VietQR + polling + xác nhận qua webhook SePay — xem Mục 8.6). Nhóm nên xác nhận lại với yêu cầu đề bài/giảng viên trước khi viết test case riêng cho "thanh toán MoMo" như một cổng thanh toán độc lập.

8. Luồng nghiệp vụ chính

Mục này mô tả từng bước xử lý (backend + frontend + bên thứ ba nếu có) của 8 luồng nghiệp vụ quan trọng nhất, làm cơ sở thiết kế test case theo kịch bản (scenario-based) và test case luồng thay thế (alternate flow).

8.1 Đăng ký / Đăng nhập bằng OTP

Bước 1: Người dùng nhập số điện thoại; frontend gọi check-account để xác định tài khoản đã tồn tại hay chưa (quyết định gửi OTP loại REGISTER hay LOGIN).

Bước 2: Backend gửi mã OTP gồm 6 chữ số tới số điện thoại, lưu bản ghi otp_verifications với hạn dùng 5 phút; giới hạn tối đa 10 lượt gửi/số điện thoại/ngày (OtpLockService, Redis).

Bước 3: Người dùng nhập mã OTP; backend so khớp với bản ghi còn hạn và chưa dùng (is_used = false).

Bước 4: Nhập sai tối đa 3 lần liên tiếp sẽ bị khoá xác thực OTP cho số điện thoại đó trong 15 phút.

Bước 5: OTP đúng: nếu là LOGIN → đăng nhập và phát JWT; nếu là REGISTER → tạo tài khoản users mới (has_password = false) rồi phát JWT.

Bước 6: Nếu là lần xác thực OTP thành công đầu tiên của số điện thoại đó, users.phone_verified_at được ghi nhận.

8.2 Đăng nhập bằng mật khẩu

Bước 1: Người dùng nhập tài khoản (username/email/phone) và mật khẩu tại LoginPasswordPage.

Bước 2: Backend so khớp mật khẩu với password_hash bằng BCrypt.

Bước 3: Mỗi lần sai, PasswordAttemptService (Redis) tăng bộ đếm theo tài khoản; vượt ngưỡng quy định sẽ bắt buộc xác minh Google reCAPTCHA v2 ở lần thử kế tiếp, tiếp tục sai sẽ bị khoá tạm thời (khóa dần — progressive lockout).

Bước 4: Đăng nhập đúng → phát cặp access token (hạn 1 giờ) và refresh token (hạn 7 ngày).

Bước 5: Frontend lưu cả hai token vào localStorage (xem Mục 6.4 về cách hai token này được dùng và tự làm mới).

8.3 Đăng nhập bằng Google OAuth2

Bước 1: Frontend hiển thị nút đăng nhập Google (@react-oauth/google), nhận về idToken từ Google sau khi người dùng xác thực.

Bước 2: Frontend gửi idToken tới POST /api/auth/google.

Bước 3: Backend xác minh idToken bằng Google API Client (GoogleTokenVerifier) — kiểm tra chữ ký và audience.

Bước 4: Nếu social_accounts đã có bản ghi khớp (provider=GOOGLE, provider_id=mã định danh Google) → đăng nhập vào tài khoản liên kết sẵn.

Bước 5: Nếu chưa có → tự động tạo mới users (lấy email/tên từ Google) và social_accounts tương ứng, rồi đăng nhập.

Bước 6: Phát JWT như luồng đăng nhập thông thường.

8.4 Đặt hàng & áp dụng khuyến mãi (Checkout)

Bước 1: CheckoutPage tải giỏ hàng hiện tại (useCartStore) và danh sách địa chỉ đã lưu của người dùng.

Bước 2: Người dùng chọn/nhập địa chỉ giao hàng; nếu địa chỉ có district_id/ward_code (đã đồng bộ GHN) → gọi API tính phí ship thời gian thực theo cân nặng; nếu không → dùng mức phí cố định 30.000đ.

Bước 3: Người dùng có thể nhập mã khuyến mãi; backend kiểm tra: còn hiệu lực (starts_at/ends_at, is_active), đạt min_order_value, chưa vượt max_uses và max_uses_per_user (đối chiếu promotion_usages), và nếu target=PERSONAL thì mã chỉ áp dụng được cho đúng user_id được gán.

Bước 4: Người dùng chọn phương thức thanh toán (COD / VNPay / MoMo / Chuyển khoản ngân hàng) rồi xác nhận đặt hàng.

Bước 5: Backend tạo orders (sinh order_code định dạng GJH-XXXXXXX, snapshot shipping_address dạng JSON), tạo order_items (snapshot tên/giá từng sản phẩm), trừ tồn kho product_variants.stock_qty, và nếu có áp dụng khuyến mãi thì ghi promotion_usages + tăng promotions.used_count.

Bước 6: Nếu chọn thanh toán online (VNPay/MoMo/Chuyển khoản) → orders.expires_at được gán hạn thanh toán; đơn sẽ tự huỷ nếu quá hạn mà chưa thanh toán (xem Mục 8.7).

8.5 Thanh toán VNPay

Bước 1: Frontend gọi API tạo URL thanh toán (yêu cầu JWT hợp lệ).

Bước 2: Backend (VnpayServiceImpl) dựng URL thanh toán, ký bằng HMAC-SHA512 với hash secret cấu hình riêng (VnpayProperties), rồi chuyển hướng người dùng sang cổng VNPay.

Bước 3: Người dùng hoàn tất thanh toán trên giao diện VNPay.

Bước 4: VNPay gọi 2 kênh callback độc lập, cả hai đều permitAll trong SecurityConfig: (a) IPN — server-to-server, backend xác minh chữ ký rồi cập nhật payment_status = PAID, đây là nguồn xác nhận chính thức; (b) Return — trình duyệt người dùng được chuyển hướng thẳng về domain Frontend (Vercel) tại /payment/vnpay/result, chỉ để hiển thị kết quả, KHÔNG dùng để xác nhận thanh toán.

Bước 5: Thiết kế redirect thẳng về Frontend (thay vì Backend tự xử lý redirect) xuất phát từ việc Frontend (Vercel) và Backend (Railway) khác domain (xem Mục 2).

8.6 Thanh toán "chuyển khoản / MoMo" (giả lập qua VietQR + SePay)

Bước 1: Người dùng chọn phương thức Chuyển khoản ngân hàng hoặc MoMo — cả hai hiện dùng chung một luồng xử lý phía backend (xem Mục 7).

Bước 2: Frontend hiển thị mã VietQR động (BankTransferModal hoặc MomoTransferModal) chứa số tiền và nội dung chuyển khoản gồm mã đơn hàng (GJH-XXXXXXX).

Bước 3: Frontend polling (hỏi lại) trạng thái đơn mỗi 3 giây để phát hiện thanh toán đã được xác nhận hay chưa.

Bước 4: Người dùng chuyển khoản thật qua ứng dụng ngân hàng của mình.

Bước 5: SePay phát hiện giao dịch vào tài khoản nhận tiền, gọi webhook tới /api/webhooks (xác thực bằng header Authorization: apikey {key} riêng, không dùng JWT).

Bước 6: SePayWebhookServiceImpl dùng regex khớp mã đơn hàng GJH-XXXXXXX trong nội dung chuyển khoản, đối chiếu số tiền, rồi cập nhật payment_status = PAID; endpoint luôn trả HTTP 200 cho SePay (kể cả khi không khớp được đơn) để tránh bị SePay gọi lại liên tục.

Bước 7: Frontend polling phát hiện trạng thái đã PAID → chuyển sang hiển thị OrderSuccessModal.

8.7 Vòng đời đơn hàng sau khi đặt

Bước 1: Đơn được tạo mới ở trạng thái PENDING.

Bước 2: Nhân viên/Admin xác nhận đơn (AdminOrderController) → chuyển CONFIRMED, rồi chuyển tiếp SHIPPING khi bàn giao cho đơn vị vận chuyển.

Bước 3: Khách hàng chỉ được huỷ đơn khi đơn đang ở PENDING; hành động "xác nhận đã nhận hàng" chỉ áp dụng được khi đơn đang ở SHIPPING → chuyển thành DELIVERED.

Bước 4: Nếu đơn thanh toán online (không phải COD) ở PENDING quá hạn expires_at → AutoCancelScheduler (chạy mỗi 5 phút) tự động chuyển CANCELLED, cancelled_by = SYSTEM, đồng thời hoàn kho và giải phóng lượt dùng khuyến mãi đã áp dụng (nếu có).

Bước 5: Nếu đơn ở SHIPPING quá 7 ngày mà chưa được cập nhật trạng thái → AutoDeliverScheduler (chạy 2:00 sáng mỗi ngày) tự động chuyển DELIVERED (và tự đánh dấu PAID nếu là COD chưa thanh toán).

8.8 Đánh giá sản phẩm sau khi nhận hàng

Bước 1: Người dùng vào lịch sử đơn hàng, chọn một sản phẩm trong đơn để đánh giá.

Bước 2: Backend ràng buộc mỗi (product_id, user_id, order_id) chỉ được đánh giá đúng 1 lần (UNIQUE 3 cột ở bảng reviews) — cùng sản phẩm nhưng mua ở đơn khác thì được đánh giá lại.

Bước 3: Đánh giá gồm rating (bắt buộc, 1-5), comment và ảnh minh hoạ tùy chọn (upload qua Cloudinary — xem Mục 6.5).

Bước 4: Đánh giá hiển thị công khai NGAY khi tạo (is_approved mặc định TRUE) — không qua bước duyệt trước của Admin/Staff.

Bước 5: Admin/Staff có thể phản hồi (reply) đánh giá; products.avg_rating và review_count được tính lại (dữ liệu tổng hợp — xem Mục 4.3).

9. Bảo mật hệ thống

Tổng hợp các cơ chế bảo mật đã khảo sát được (chi tiết từng phần tham chiếu tới các mục tương ứng ở trên):

- Xác thực JWT stateless: access token hạn 1 giờ, refresh token hạn 7 ngày; token bị thu hồi khi logout được đưa vào danh sách đen (blacklist) lưu ở Redis (Mục 5.2).
- Phân quyền theo vai trò (CUSTOMER/STAFF/ADMIN) bằng @PreAuthorize khai báo riêng trên từng controller/method — không tập trung ở SecurityConfig; có 1 ngoại lệ đã ghi nhận là thiếu ràng buộc vai trò (AdminController, Mục 11.1).
- Mật khẩu băm bằng BCrypt; chống brute-force đăng nhập mật khẩu bằng cơ chế khoá dần + Google reCAPTCHA v2 sau ngưỡng số lần sai (Mục 8.2).
- Chống brute-force OTP: giới hạn số lượt gửi/ngày và số lần nhập sai trước khi khoá tạm theo số điện thoại (Mục 8.1).
- CORS: danh sách origin được phép khai báo tường minh (CorsConfig), ngoại trừ /api/webhooks/** vốn cần mở cho bên thứ ba (SePay, VNPay IPN) gọi vào — các endpoint này tự bảo vệ bằng cơ chế xác thực riêng (API key hoặc chữ ký) thay vì JWT.
- CSRF được tắt hoàn toàn vì hệ thống không dùng cookie/session để xác thực — toàn bộ xác thực qua Bearer token trong header Authorization.
- Token JWT (access + refresh) được lưu ở localStorage phía trình duyệt, không phải cookie HttpOnly (Mục 6.4) — đặc điểm kiến trúc cần cân nhắc khi kiểm thử bảo mật (bề mặt XSS lý thuyết cao hơn).
- Giá trị bí mật (khoá JWT, mật khẩu ứng dụng Gmail, secret reCAPTCHA, token GHN, hash secret VNPay, khoá SePay) được tách riêng theo Spring Profile (application-local.yml) — ngoài phạm vi trích dẫn của tài liệu này.

10. Triển khai

- Frontend: triển khai trên Vercel (static hosting + CDN), build bằng Vite.
- Backend: triển khai trên Railway dạng container, build bằng Maven (pom.xml); có hiện tượng "cold start" (khởi động lại chậm) khi không có traffic một thời gian — frontend đã có cơ chế tự retry sau 12 giây khi gặp lỗi mạng ở request đầu tiên (Mục 6.4).
- Cơ sở dữ liệu & cache: MySQL 8 và Redis, chạy trên hạ tầng phù hợp với Railway hoặc dịch vụ ngoài tương đương.
- Frontend (Vercel) và Backend (Railway) nằm ở 2 domain khác nhau → CORS phải khai báo rõ origin được phép; các luồng cần redirect quay lại trình duyệt sau khi rời hệ thống (VNPay Return) phải trở thẳng về domain Frontend thay vì Backend (Mục 2, Mục 8.5).
- Biến môi trường phía frontend dùng tiền tố VITE_ (ví dụ VITE_API_URL, VITE_CLOUDINARY_CLOUD_NAME) theo quy ước chuẩn của Vite — được nhúng vào bundle tại thời điểm build, không phải runtime.

11. Hạn chế đã biết & khuyến nghị cải thiện

Đây là tổng hợp các điểm mã nguồn thể hiện chưa rõ ràng, có khả năng chưa nhất quán, hoặc là phát hiện đáng chú ý trong quá trình khảo sát — nhóm nên ưu tiên xác nhận lại bằng kiểm thử thực tế trước khi kết luận là lỗi chính thức, và đưa vào kế hoạch kiểm thử (test plan) ở mức độ ưu tiên phù hợp.

11.1 AdminTagController thiếu ràng buộc theo vai trò

Trong số 9 controller thuộc khu vực `/api/admin/**`, chỉ riêng AdminTagController không có annotation `@PreAuthorize`. Vì SecurityConfig chỉ yêu cầu "đã đăng nhập" (`anyRequest().authenticated()`) chứ không tự kiểm tra vai trò, nên 4 endpoint quản lý tag (`/api/admin/tags`) hiện có thể bị gọi bởi BẤT KỲ tài khoản nào đã đăng nhập — kể cả một CUSTOMER thông thường, không chỉ riêng STAFF/ADMIN.

Khuyến nghị / Cần kiểm thử: Viết test case đăng nhập bằng tài khoản CUSTOMER và gọi trực tiếp 4 endpoint `/api/admin/tags` để xác nhận có bị chặn hay không (kỳ vọng: 403, thực tế nghi ngờ: 200).

11.2 Hai DTO/endpoint đổi mật khẩu trùng lặp

Mã nguồn có 2 lớp DTO cùng tên `ChangePasswordRequest` ở 2 domain khác nhau (`dto/auth/request` và `dto/user/request`), tương ứng khả năng tồn tại 2 endpoint đổi mật khẩu riêng biệt (một thuộc `AuthController`, một thuộc `UserController`).

Khuyến nghị / Cần kiểm thử: Xác nhận cả 2 endpoint có cùng logic validate và cùng tác dụng phụ (có vô hiệu hoá các phiên đăng nhập khác hay không) hay không, tránh trường hợp một endpoint kiểm soát chặt còn endpoint kia bị bỏ sót ràng buộc.

11.3 "MoMo" không phải tích hợp API MoMo thật

`payments.method` có giá trị MOMO riêng biệt với `BANK_TRANSFER` ở tầng CSDL, nhưng ở tầng xử lý, cả hai hiện dùng chung cơ chế mô phỏng: hiển thị mã VietQR ngân hàng + xác nhận qua webhook SePay (Mục 8.6) — không có gọi API chính thức của MoMo.

Khuyến nghị / Cần kiểm thử: Làm rõ với yêu cầu đề bài/giảng viên xem đây có phải phạm vi chấp nhận được hay cần bổ sung tích hợp MoMo thật trước khi viết test case coi "thanh toán MoMo" là một cổng thanh toán độc lập.

11.4 GHN chỉ hỗ trợ giao hàng nội thành Hồ Chí Minh

Mảng `DELIVERY_PROVINCES` ở frontend (`LocationSelect.jsx`) hiện chỉ chứa "Hồ Chí Minh" — người dùng ở tỉnh/thành khác không chọn được địa chỉ giao hàng qua luồng GHN chuẩn. Khi địa chỉ thiếu `district_id/ward_code`, hệ thống dùng mức phí ship cố định 30.000đ thay vì tính theo GHN.

Khuyến nghị / Cần kiểm thử: Kiểm thử cả 2 nhánh: đặt hàng với địa chỉ trong TP.HCM (tính phí GHN thời gian thực) và đặt hàng với địa chỉ ngoài TP.HCM/không có `district_id-ward_code` (phí cố định 30.000đ).

11.5 `product_tags.tag` không ràng buộc khoá ngoại

Cột `product_tags.tag` là `VARCHAR(50)` tự do, không có khoá ngoại tới `tag_definitions.name` — CSDL không ngăn việc gán một giá trị tag bất kỳ, kể cả không tồn tại trong danh mục chuẩn `tag_definitions`.

Khuyến nghị / Cần kiểm thử: Kiểm thử tạo/sửa sản phẩm qua API với giá trị tag tùy ý (không có trong tag_definitions) để xác nhận validate có được thực hiện ở tầng ứng dụng (service/DTO) hay hoàn toàn không có ràng buộc.

11.6 Chưa rõ khuôn dạng phản hồi lỗi 403 do phân quyền

GlobalExceptionHandler không có @ExceptionHandler riêng cho AccessDeniedException (lỗi phát sinh khi @PreAuthorize chặn một request đã đăng nhập nhưng sai vai trò) — cơ chế xử lý mặc định của Spring Security cho loại lỗi này hoạt động độc lập với @RestControllerAdvice.

Khuyến nghị / Cần kiểm thử: Gọi thử 1 endpoint có @PreAuthorize bằng tài khoản sai vai trò và so sánh khuôn dạng JSON trả về với khuôn dạng chuẩn {status, message, timestamp} dùng cho các lỗi khác trong hệ thống.

11.7 Token JWT lưu ở localStorage thay vì cookie HttpOnly

accessToken và refreshToken được frontend lưu trong localStorage (axiosConfig.js), có thể bị đọc bởi bất kỳ đoạn JavaScript nào chạy được trên trang — bề mặt tiếp xúc XSS lý thuyết cao hơn so với phương án cookie HttpOnly.

Khuyến nghị / Cần kiểm thử: Đây là đặc điểm kiến trúc, không nhất thiết là lỗi — nhóm nên đưa vào phạm vi kiểm thử bảo mật (security testing) nếu đề bài yêu cầu, thay vì coi là chức năng cần sửa.

11.8 SettingsPage: 2 chức năng chưa nối API backend

Chức năng xóa tài khoản và tùy chọn nhận thông báo tại trang Cài đặt hiện là placeholder ở giao diện — chưa gọi API backend tương ứng.

Khuyến nghị / Cần kiểm thử: Kiểm thử xác nhận hành vi thực tế khi bấm các nút này (không có tác dụng / báo lỗi / giả lập thành công) để mô tả đúng vào test case, tránh báo lỗi nhầm cho một chức năng vốn chưa được cài đặt.

11.9 Yêu cầu phiên bản Java chưa nhất quán với README

README của dự án ghi yêu cầu Java ≥ 17 , nhưng cấu hình build thực tế (pom.xml, Spring Boot 4.0.6) đòi hỏi Java 21 mới build thành công (Mục 3.2).

Khuyến nghị / Cần kiểm thử: Nhóm cần thống nhất môi trường build/CI dùng đúng Java 21 khi thiết lập môi trường kiểm thử, tránh mất thời gian debug lỗi build do sai phiên bản JDK.

11.10 Thiếu tài liệu API tự sinh (Swagger/OpenAPI)

pom.xml không có springdoc-openapi hoặc Swagger — hệ thống chưa có trang tài liệu API tự sinh nào để tra cứu nhanh request/response mẫu.

Khuyến nghị / Cần kiểm thử: Nhóm nên tự xây dựng và duy trì 1 bộ Postman Collection dùng chung (đã có MCP Postman sẵn trong công cụ hỗ trợ) làm tài liệu tham chiếu thay thế trong suốt quá trình kiểm thử.

11.11 AutoDeliverScheduler suy đoán giao hàng thành công theo thời gian

AutoDeliverScheduler tự động chuyển đơn từ SHIPPING sang DELIVERED chỉ dựa trên mốc 7 ngày kể từ lần cập nhật cuối, không có bước xác nhận thực tế từ đơn vị vận chuyển GHN rằng đơn đã giao thành công.

Khuyến nghị / Cần kiểm thử: Kiểm thử biên đúng tại mốc 7 ngày (6 ngày 23 giờ vs. 7 ngày 1 giờ), và đánh giá rủi ro nghiệp vụ nếu đơn COD bị tự động đánh dấu DELIVERED + PAID trong khi thực tế giao hàng thất bại.

11.12 JwtAuthFilter bỏ qua hoàn toàn xác thực trên /api/auth/logout

JwtAuthFilter cấu hình `shouldNotFilter()` để bỏ qua hoàn toàn việc phân tích token trên 2 đường dẫn /api/auth/refresh và /api/auth/logout — nghĩa là request logout không đi qua bước trích xuất token bằng bộ lọc chuẩn.

Khuyến nghị / Cần kiểm thử: Xác minh cơ chế thực tế: endpoint logout xác định access token nào cần đưa vào danh sách đen bằng cách nào (nhận qua body request, hay tự parse header thủ công trong AuthController) — viết test case logout rồi thử dùng lại access token cũ để xác nhận token đã thật sự bị vô hiệu hoá.

12. Phụ lục

12.1 Số lượng file mã nguồn Backend theo package

Package	Số file	Ghi chú
config/ (gồm config/properties/)	10	4 lớp cấu hình chính + 6 lớp *Properties
controller/	26	124 endpoint tổng cộng — Phụ lục A, Tài liệu SRS
dto/**/request, response	75	Tách riêng request/response theo domain
entity/	23	Tương ứng 1-1 với 23 bảng CSDL — Mục 4
exception/	2	AppException, GlobalExceptionHandler
mapper/	10	Chuyển đổi entity ↔ DTO thủ công
middleware/	1	JwtAuthFilter
repository/	23	Tương ứng 1-1 với entity
service/ (gồm impl/)	54	16 domain nghiệp vụ
utils/	1	JwtUtil
(gốc) BackendApplication.java	1	Lớp khởi động Spring Boot
Tổng cộng	226	File .java trong backend/src/main/java

12.2 Số lượng file mã nguồn Frontend theo thư mục

Thư mục	Số file	Ghi chú
api/	21	20 module gọi REST API theo domain + axiosConfig.js
components/	29	Chia theo domain: cart, common, layout, order, product, user
hooks/	7	Custom hook dùng chung
pages/	34	Chia theo domain, gồm cả khu vực admin/
store/	5	Store Zustand — Mục 6.3
Khác ở gốc src/	11	App.jsx, App.css, main.jsx, index.css, constants/, utils/, assets/ (4 file ảnh/svg)
Tổng cộng	107	File trong frontend/src

12.3 Bảng thuật ngữ viết tắt

Viết tắt	Giải thích
SRS	Software Requirements Specification — Tài liệu đặc tả yêu cầu phần mềm
SDD	System Description Document — tài liệu này
FR	Functional Requirement — Yêu cầu chức năng (đánh số trong Tài liệu SRS)
RBAC	Role-Based Access Control — Kiểm soát truy cập theo vai trò
JWT	JSON Web Token — chuẩn token dùng cho xác thực stateless
OTP	One-Time Password — mã dùng một lần

Viết tắt	Giải thích
DTO	Data Transfer Object — đối tượng dùng để truyền dữ liệu qua API
ORM	Object-Relational Mapping — ánh xạ đối tượng ↔ quan hệ CSDL (ở đây là JPA/Hibernate)
GHN	Giao Hàng Nhanh — đơn vị vận chuyển tích hợp
IPN	Instant Payment Notification — cơ chế callback server-to-server của VNPay
CORS	Cross-Origin Resource Sharing — cơ chế trình duyệt kiểm soát gọi API khác domain
CSRF	Cross-Site Request Forgery — một dạng tấn công giả mạo yêu cầu
XSS	Cross-Site Scripting — một dạng tấn công chèn mã kịch bản
TTL	Time To Live — thời gian sống của dữ liệu tạm (dùng cho khoá Redis)
BCrypt	Thuật toán băm mật khẩu một chiều, có độ trễ tính toán cố ý để chống brute-force
reCAPTCHA v2	Dịch vụ xác minh không phải robot của Google, dùng chặn đăng nhập tự động